Apontadores

Programação e Desenvolvimento de Software I
Heitor Ramos

Definição

- Variável
  - É um espaço reservado de memória usado para guardar um ***valor*** que pode ser modificado pelo programa;

- Ponteiro
  - É um espaço reservado de memória usado para guardar o ***endereço de memória*** de uma outra variável.
  - Um ponteiro é uma variável como qualquer outra do programa – sua diferença é que ela não armazena um valor inteiro, real, caractere ou booleano.
  - Ela serve para armazenar endereços de memória (são valores inteiros sem sinal).

Declaração

- Como qualquer variável, um ponteiro também possui um tipo

```
//declaração de variável
tipo_variável  nome_variável;

//declaração de ponteiro
tipo_ponteiro *nome_ponteiro;
```

- É o ***asterisco*** (*) que informa ao compilador que aquela variável não vai guardar um valor mas sim um endereço para o tipo especificado.

```
int x;
float y;
struct ponto p;

int *x;
float *y;
struct ponto *p;
```

Declaração

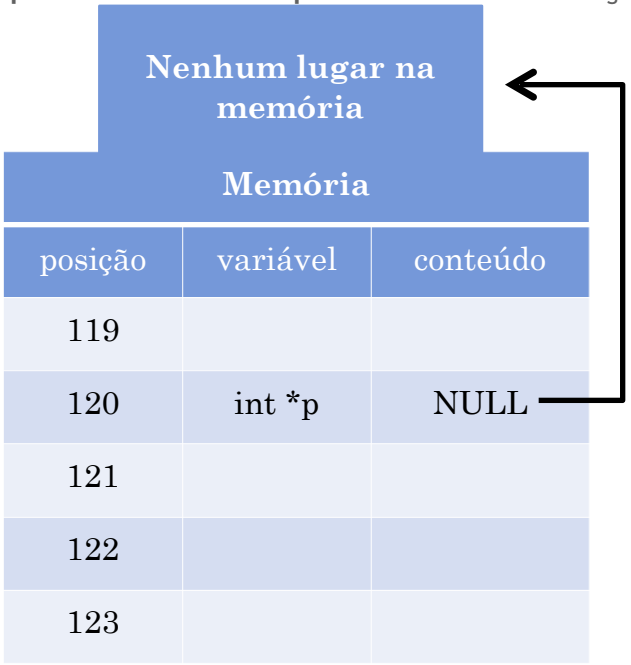
- Exemplos de declaração de variáveis e ponteiros

```
int main(){
    //Declara um ponteiro para int
    int *p;
    //Declara um ponteiro para float
    float *x;
    //Declara um ponteiro para char
    char *y;
    //Declara um ponteiro para struct ponto
    struct ponto *p;
    //Declara uma variável do tipo int e um ponteiro para int
    int soma, *p2,;

    return 0;
}
```

Inicialização

- Um ponteiro pode ter o valor especial NULL que é o endereço de nenhum lugar.
- Exemplo
  - `int *p = NULL;`



Inicialização

- Os ponteiros devem ser inicializados antes de serem usados.
- Assim, devemos apontar um ponteiro para um lugar conhecido
  - Podemos apontá-lo para uma variável que já exista no programa.

Memória		
posição	variável	conteúdo
119		
120	int *p	122
121		
122	int c	10
123		

Utilização

- Para acessar o **valor** guardado dentro de uma posição na memória apontada por um ponteiro, basta usar o operador ***asterisco*** "*" na frente do nome do ponteiro

```
int main(){
    //Declara uma variável int contendo o valor 10
    int c = 10;
    //Declara um ponteiro para int
    int *p;
    //Atribui ao ponteiro o endereço da variável int
    p = &c;
    printf("Conteudo apontado por p: %d \n",*p);// 10
    //Atribui um novo valor à posição de memória apontada por p
    *p = 12;
    printf("Conteudo apontado por p: %d \n",*p);// 12
    printf("Conteudo de count: %d \n",c);// 12

    return 0;
}
```

Operações com ponteiros

- Atribuição
  - p1 aponta para o mesmo lugar que p2;

```
int *p, *p1;
int c = 10;
p = &c;
p1 = p;//equivale a p1 = &c;
```

  - a variável apontada por p1 recebe o mesmo conteúdo da variável apontada por p2;

```
int *p, *p1;
int c = 10, d = 20;
p = &c;
p1 = &d;

*p1 = *p;//equivale a d = c;
```

Operações com ponteiros

- Apenas duas operações aritméticas podem ser utilizadas com no endereço armazenado pelo ponteiro: adição e subtração
- podemos apenas somar e subtrair valores INTEIROS
  - p++;
    - soma +1 no endereço armazenado no ponteiro.
  - p--;
    - subtrai 1 no endereço armazenado no ponteiro.
  - p = p+15;
    - soma +15 no endereço armazenado no ponteiro.

Operações com ponteiros

- As operações de adição e subtração no endereço dependem do tipo de dado que o ponteiro aponta.
  - Considere um ponteiro para inteiro, int *.
  - O tipo int ocupa um espaço de 4 bytes na memória.
  - Assim, nas operações de adição e subtração são adicionados/subtraídos 4 bytes por incremento/decremento, pois esse é o tamanho de um inteiro na memória e, portanto, é também o valor mínimo necessário para sair dessa posição reservada de memória

Memória		
posição	variável	conteúdo
119		
120	int a	10
121		
122		
123		
124	int b	20
125		
126		
127		
128	char c	‘k’
129	char d	‘s’
130		

Operações com ponteiros

- Operações Ilegais com ponteiros
  - Dividir ou multiplicar ponteiros;
  - Somar o endereço de dois ponteiros;
  - Não se pode adicionar ou subtrair valores dos tipos **float** ou **double** de ponteiros.

Operações com ponteiros

- Já sobre seu conteúdo apontado, valem todas as operações
  - (*p)++;
    - incrementar o conteúdo da variável apontada pelo ponteiro p;
  - *p = (*p) * 10;
    - multiplica o conteúdo da variável apontada pelo ponteiro p por 10;

```
int *p;
int c = 10;

p = &c;

(*p)++;
*p = (*p) * 10;
```

Operações com ponteiros

- Operações relacionais
  - **==** e **!=** para saber se dois ponteiros são iguais ou diferentes.
  - **>**, **<**, **>=** e **<=** para saber qual ponteiro aponta para uma posição mais alta na memória.

```
int main(){
    int *p, *p1, x, y;
    p = &x;
    p1 = &y;
    if(p == p1)
        printf("Ponteiros iguais\n");
    else
        printf("Ponteiros diferentes\n");

    return 0;
}
```

Relembrando Ponteiros

& ➔ retorna o endereço de memória de uma variável. Em outras palavras, um ponteiro

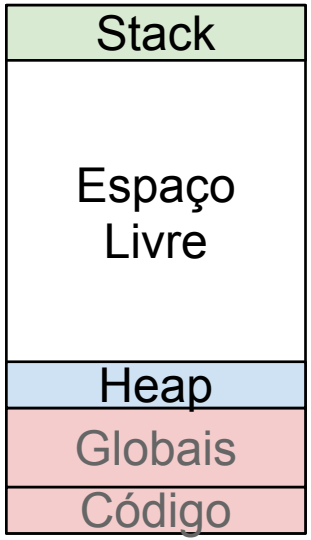
* ➔ utilizado para indicar ponteiros

```
// y é uma variável com valor 10
// y = 10; y++ = 11;
int  y = 10;

// y_ptr uma variável com o valor do endereço de y
// y_ptr = 0x7fff6b481f90; y_ptt++ = 0x7fff6b481f94;
int *y_ptr = &y;
```

Esquema de um Programa na Memória

- Podemos simplificar um programa com uma esquema como o abaixo
- [Sempre] Código
  - Que ocupa espaço
- [Pode ter] Variáveis globais
  - Que ocupa espaço
- [Sempre] Stack ou Pilha
  - Utilizado pelo hardware para chamar funções
- [Sempre] Heap
  - Alocação dinâmica



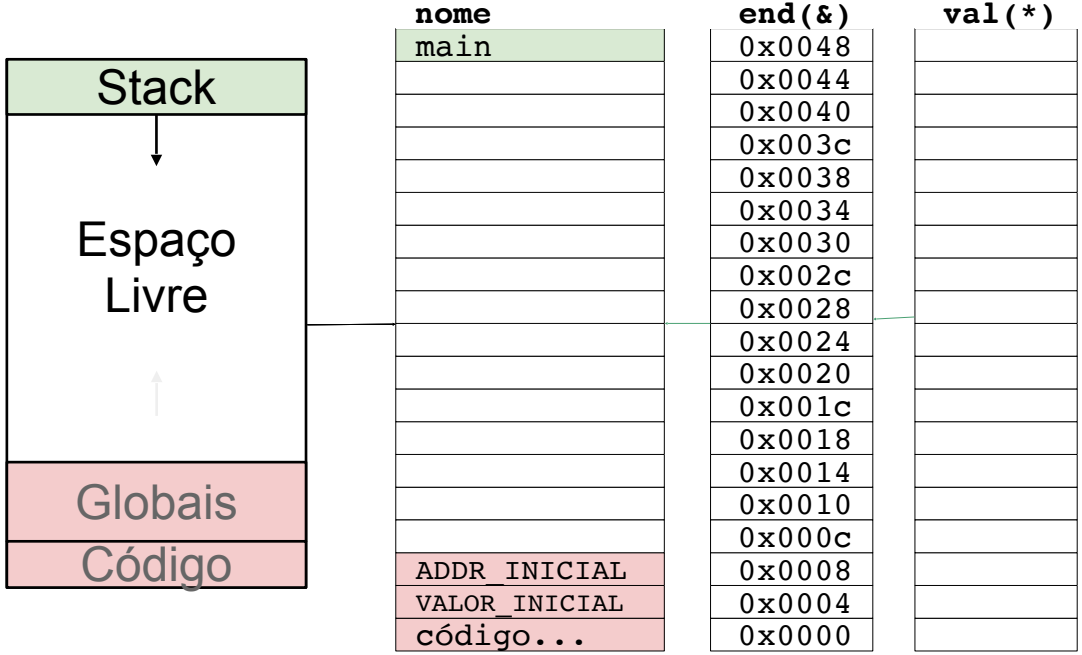
Exemplo

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



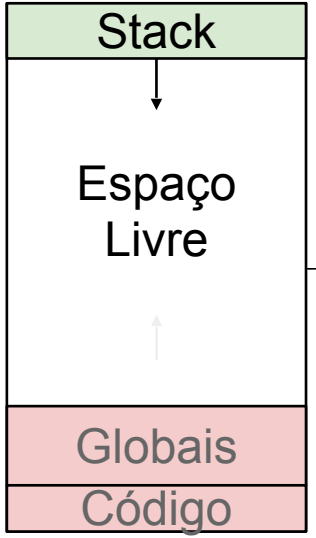
Exemplo

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
	0x0044	
	0x0040	
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

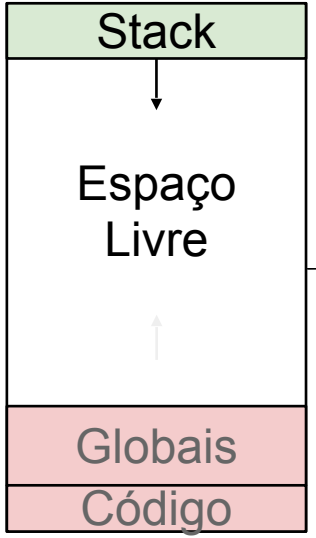
Exemplo

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	7
	0x0040	
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

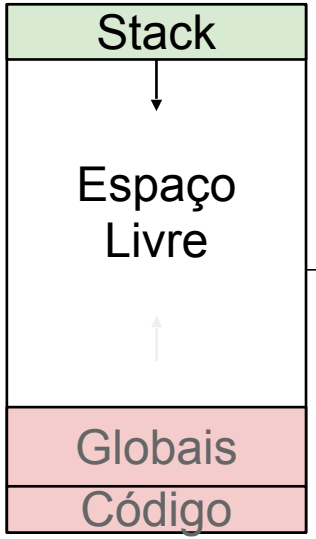
Exemplo

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	**0x0044**	7
*r	0x0040	**0x0044**
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

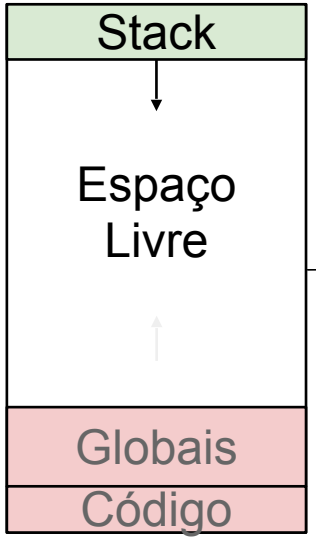
Exemplo

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	18
*r	0x0040	0x0044
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

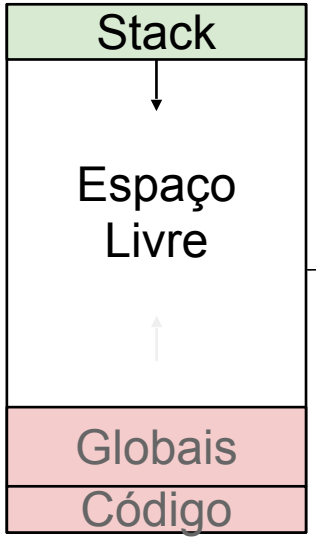
```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```

44+2C, onde 2C = (4*11) em hexa



nome	end(&)	val(*)
main	0x0048	
a	0x0044	18
*r	0x0040	0x0070
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

bem esquisito!

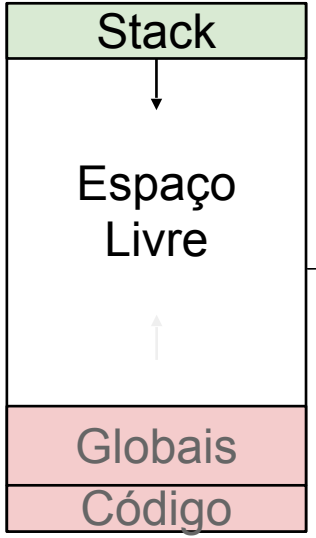
por motivos similares que o programa quebra

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	18
*r	**0x0040**	**0x0070**
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

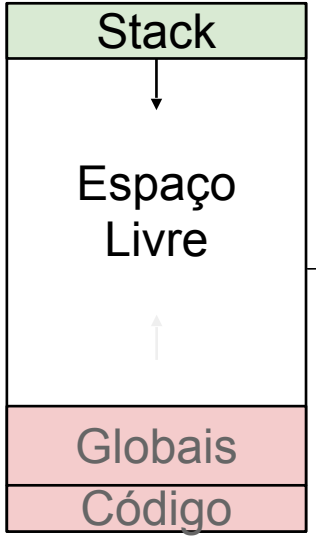
chamando uma função

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	8
*r	**0x0040**	**0x0070**
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

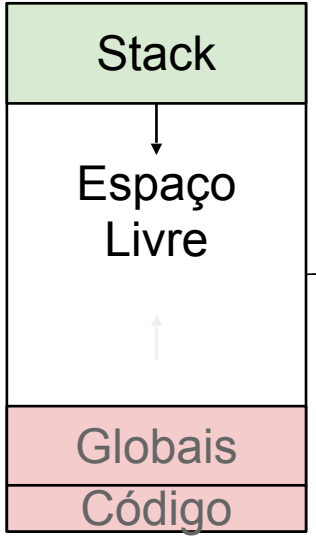
a pilha cresce e temos espaço para a mesma

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	8
*r	0x0040	0x0070
pglobais	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

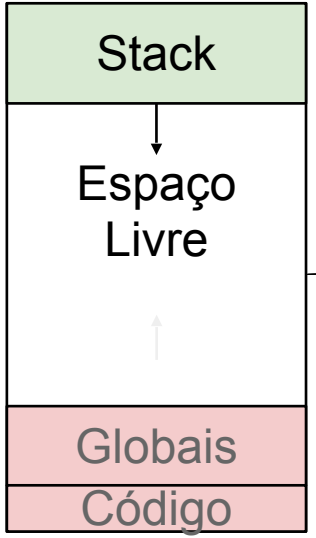
a pilha cresce e temos espaço para a mesma

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	8
*r	**0x0040**	**0x0070**
pglobais	0x003c	
sem_uso	0x0038	99
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

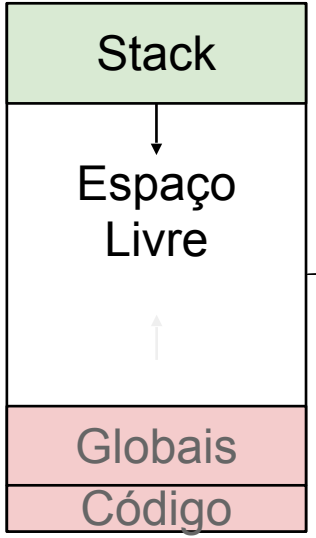
no fim volta ao normal

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	8
*r	**0x0040**	**0x0070**
pglobais	0x003c	
sem_uso	0x0038	99
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

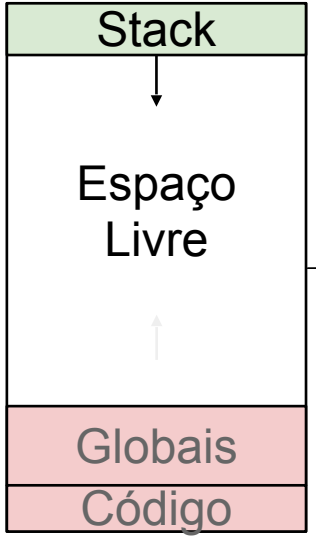
no fim volta ao normal

```c
#include <stdio.h>

const int VAL_GLB = 7;
const int *END_GLB = &VAL_GLB;

void pglobais(void) {
   int sem_uso = 99;
   printf("end glb %p\n", END_GLB);
   printf("val glb %d\n", VAL_GLB);
}

int main(void) {
  int a = VAL_GLB;
  int *r = &a;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  a+=11;
  r+=11;
  printf("end loc %p\n", r);
  printf("val loc %d\n", a);
  pglobais();
  return 0;
}
```



nome	end(&)	val(*)
main	0x0048	
a	0x0044	8
*r	**0x0040**	**0x0070**
	0x003c	
	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
ADDR_INICIAL	0x0008	0x0004
VALOR_INICIAL	0x0004	7
código...	0x0000	

Stack/Pilha

Então sabemos o que é a pilha. Onde as "funções crescem"

No esquema abaixo desenhamos a pilha crescendo para baixo

O oposto também é comum (depende do livro/arquitetura)

Ponteiro para Struct

```
typedef struct {
  double x;
  double y;
} ponto_t;

int main(void) {
  ponto_t ponto;
  ponto_t *ponto_ptr = &ponto;
  ponto_ptr->x = 20;  // equivalente a ponto.x. -> é um “atalho” (*ponto_ptr).x
  ponto_ptr->y = 99;  // equivalente a ponto.y. -> é um “atalho" (*ponto_ptr).y
  return 0;
}
```

Função para Iniciar um ponto...

```
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", &p->x);            // antes era (*p).x
  printf("Digite a coord y: ");
  scanf("%lf", &p->y);             // antes era (*p).y
}
```

Apontadores II

Programação e Desenvolvimento de Software I
Heitor Ramos

Ponteiro Genérico

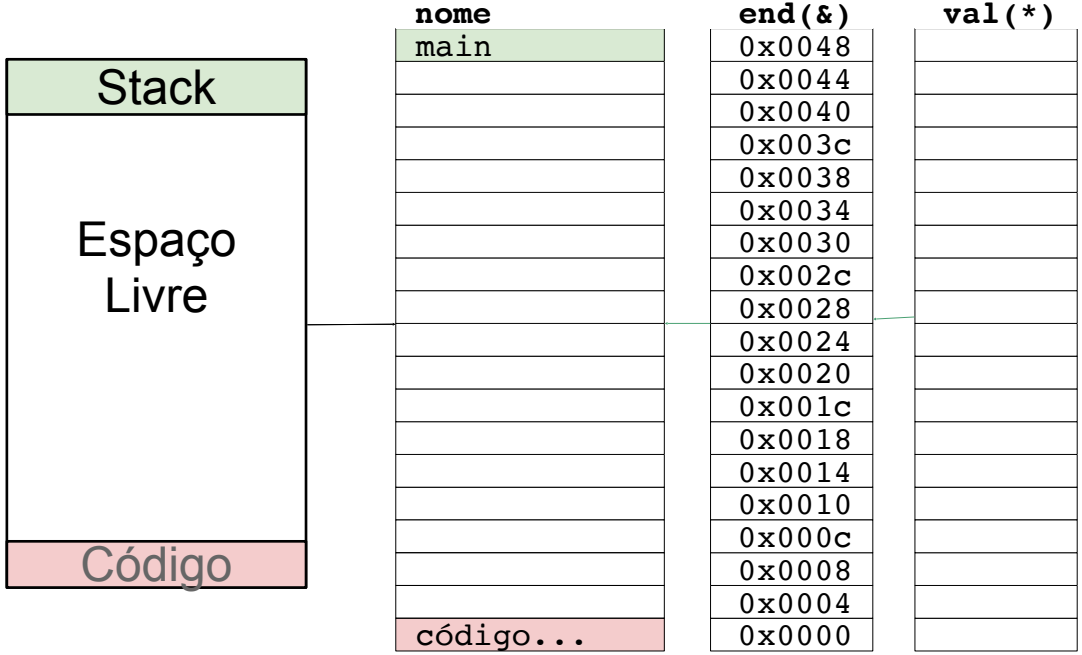
- void *p
- Mostrar exemplo do dump

Passagem por Referência (novamente)

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```



note que o ponto ocupa 4 locais (2 cada double)

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
ponto.x	0x0044	??
ponto.x	0x0040	
ponto.y	0x003c	??
ponto.y	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
	0x0008	
	0x0004	
código...	0x0000	

chamamos a função

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
ponto.x	0x0044	??
ponto.x	0x0040	
ponto.y	0x003c	??
ponto.y	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
	0x0008	
	0x0004	
código...	0x0000	

chamamos a função

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
ponto.x	0x0044	??
ponto.x	0x0040	
ponto.y	0x003c	??
ponto.y	0x0038	
inicia_ponto	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
	0x0008	
	0x0004	
código...	0x0000	

recebeu um ponteiro para ponto

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
ponto.x	0x0044	??
ponto.x	0x0040	
ponto.y	0x003c	??
ponto.y	0x0038	
inicia_ponto	0x0034	
*p	0x0030	0x0044
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
	0x0008	
	0x0004	
código...	0x0000	

ao ler um valor atualizamos o local correto

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
ponto.x	0x0044	98
ponto.x	0x0040	
ponto.y	0x003c	??
ponto.y	0x0038	
inicia_ponto	0x0034	
*p	0x0030	0x0044
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
	0x0008	
	0x0004	
código...	0x0000	

lendo o outro valor

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
ponto.x	0x0044	98
ponto.x	0x0040	
ponto.y	0x003c	72
ponto.y	0x0038	
inicia_ponto	0x0034	
*p	0x0030	0x0044
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
	0x0008	
	0x0004	
código...	0x0000	

ao retornar a função sai da pilha

```c
typedef struct {
  double x;
  double y;
} ponto_t;

void inicia_ponto(ponto_t *p) {
  printf("Digite a coord x: ");
  scanf("%lf", p->x);
  printf("Digite a coord y: ");
  scanf("%lf", p->y);
}

int main(void) {
  ponto_t ponto;
  inicia_ponto(&ponto);
  return 0;
}
```

nome	end(&)	val(*)
main	0x0048	
ponto.x	0x0044	98
ponto.x	0x0040	
ponto.y	0x003c	72
ponto.y	0x0038	
	0x0034	
	0x0030	
	0x002c	
	0x0028	
	0x0024	
	0x0020	
	0x001c	
	0x0018	
	0x0014	
	0x0010	
	0x000c	
	0x0008	
	0x0004	
código...	0x0000	

Ponteiros e passagem de parâmetros

Passagem de Parâmetros

- Na linguagem C, os parâmetros de uma função são sempre passados por ***valor***, ou seja, uma cópia do valor do parâmetro é feita e passada para a função.
- Mesmo que esse valor mude dentro da função, nada acontece com o valor de fora da função.

Passagem por valor

```
01  include <stdio.h>
02  include <stdlib.h>
03
04  void soma _ mais _ um(int n){
05    n = n + 1;
06    printf("Dentro da funcao: x = %d\n",n);
07  }
08
09  int main(){
10    int x = 5;
11    printf("Antes da funcao: x = %d\n",x);
12    soma _ mais _ um(x);
13    printf("Depois da funcao: x = %d\n",x);
14    system("pause");
15    return 0;
16  }
```

```
Saída   Antes da funcao: x = 5
        Dentro da funcao: x = 6
        Depois da funcao: x = 5
```

Passagem por referência

- Quando se quer que o valor da variável mude dentro da função, usa-se passagem de parâmetros por ***referência***.
- Neste tipo de chamada, não se passa para a função o valor da variável, mas a sua ***referência*** (seu endereço na memória);

Passagem por referência

- Utilizando o endereço da variável, qualquer alteração que a variável sofra dentro da função será refletida fora da função. Ex: função scanf()
  - sempre que desejamos ler algo do teclado, passamos para a função **scanf()** o nome da variável onde o dado será armazenado. Essa variável tem seu valor modificado dentro da função **scanf()**, e seu valor pode ser acessado no programa principal

Passagem por referência

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  int main(){
04    int x = 5;
05    printf("Antes do scanf: x = %d\n",x);
06    printf("Digite um numero: ");
07    scanf("%d",&x);
08    printf("Depois do scanf: x = %d\n",x);
09    system("pause");
10    return 0;
11  }
```

Passagem por referência

- Para passar um parâmetro por referência, passamos o ponteiro como parâmetro:

```
float sqr (float *num);
```

- Ao se chamar a função, é necessário agora utilizar o operador “&”, igual como é feito com a função **scanf()**:

```
y = sqr(&x);
```

Passagem por referência

- No corpo da função, é necessário trabalhar corretamente com o ponteiro

Por valor

```
void soma_mais_um(int n){
  n = n + 1;
}
```

Por referência

```
void soma_mais_um(int *n){
  *n = *n + 1;
}
```

Passagem por referência

```
01      include <stdio.h>
02      include <stdlib.h>
03
04      void soma _ mais _ um(int *n){
05        *n = *n + 1;
06         printf("Dentro da funcao: x = %d\n",*n);
07      }
08
09      int main(){
10        int x = 5;
11        printf("Antes da funcao: x = %d\n",x);
12        soma _ mais _ um(&x);
13        printf("Depois da funcao: x = %d\n",x);
14        system("pause");
15        return 0;
16      }
```

```
Saída    Antes da funcao: x = 5
         Dentro da funcao: x = 6
         Depois da funcao: x = 6
```

Exercício

- Crie uma função que troque o valor de dois números inteiros passados por referência.

Exercício

```
void Troca (int*a,int*b){
  int temp;
  temp = *a;
  *a = *b;
  *b = temp;
}
void Troca (int*a,int*b){
  *a = *a^*b;
  *b = *b^*a;
  *a = *a^*b;
}
```

Arrays como parâmetros

- Para utilizar arrays como parâmetros de funções alguns cuidados simples são necessários.

Arrays como parâmetros

- Arrays são sempre passados por referência para uma função;
  - A passagem de arrays ***por referência*** evita a cópia desnecessária de grandes quantidades de dados para outras áreas de memória durante a chamada da função, o que afetaria o desempenho do programa.

Arrays como parâmetros

- É necessário declarar um segundo parâmetro (em geral uma variável inteira) para passar para a função o tamanho do array separadamente.
  - Quando passamos um array por parâmetro, independente do seu tipo, o que é de fato passado é o endereço do primeiro elemento do array.

Arrays como parâmetros

- Na passagem de um array como parâmetro de uma função podemos declarar a função de diferentes maneiras, todas equivalentes:

```
void imprime (int *m, int n);

void imprime (int m[], int n);

void imprime (int m[5], int n);
```

Arrays como parâmetros

```
void imprime (int *m,int n){
int i;
for (i=0; i< n;i++)
    printf ("%d \n", m[i]);
}

int main (){
  int n[5] = {1,2,3,4,5};
  imprime(n,5);
  return 0;
}
```

Memória

.
.
.

#	var	conteúdo
123	int *m	#125
124		
125	Int *n	1
126		2
127		3
128		4
129		5

.
.
.

Arrays como parâmetros

- Vimos que para arrays, não é necessário especificar o número de elementos para a função.

```
void imprime (int*m, int n);

void imprime (int m[], int n);
```

- No entanto, para arrays com mais de uma dimensão, é necessário especificar o tamanho de todas as dimensões, exceto a primeira

```
void imprime (int m[][5], int n);
```

Arrays como parâmetros

- Na passagem de um array para uma função, o compilador precisar saber o tamanho de cada elemento, não o número de elementos.
- Uma matriz pode ser interpretada como um array de arrays.
  - **int m[4][5]**: array de 4 elementos onde cada elemento é um array de 5 posições inteiras.

Arrays como parâmetros

- Logo, o compilador precisa saber o tamanho de cada elemento do array.

```
int m[4][5]
void imprime (int m[][5], int n);
```

- Na notação acima, informamos ao compilador que estamos passando um array, onde cada elemento dele é outro array de 5 posições inteiras.

Arrays como parâmetros

- Isso é necessário para que o programa saiba que o array possui mais de uma dimensão e mantenha a notação de um conjunto de colchetes por dimensão.

- As notações abaixo funcionam para arrays com mais de uma dimensão. Mas o array é tratado como se tivesse apenas uma dimensão dentro da função

```
void imprime (int*m, int n);
void imprime (int m[], int n);
```

**colunas

*linhas

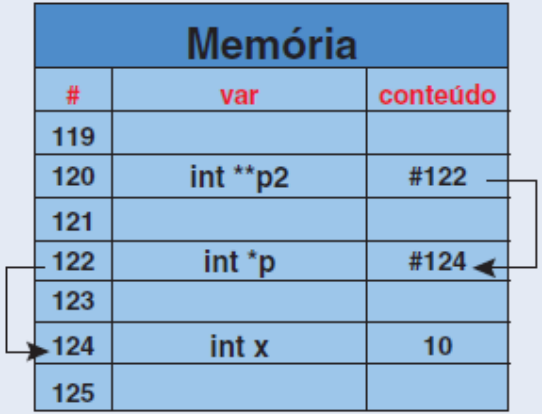
Ponteiro para ponteiro

- Um ponteiro para um ponteiro é como se você anotasse o endereço de um papel que tem o endereço da casa do seu amigo.
- Podemos declarar um ponteiro para um ponteiro com a seguinte notação
  - tipo_ponteiro **nome_ponteiro;
  - **nome_ponteiro é o conteúdo final da variável apontada;
  - *nome_ponteiro é o conteúdo do ponteiro intermediário.

Ponteiro para ponteiro

Exemplo: ponteiro para ponteiro

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  int main(){
04    int x = 10;
05    int *p = &x;
06    int **p2 = &p;
07    //Endereço em p2
08    printf("Endereco em p2: %p\n",p2);
09    //Conteudo do endereço
10    printf("Conteudo em *p2: %p\n",*p2);
11    //Conteudo do endereço do endereço
12    printf("Conteudo em **p2: %d\n",**p2);
13    system("pause");
14    return 0;
15  }
```



Ponteiro para ponteiro

- É a quantidade de asteriscos (*) na declaração do ponteiro que indica o número de níveis de apontamento que ele possui.

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  int main(){
04    //variável inteira
05    int x;
06    //ponteiro para um inteiro (1 nível)
07    int *p1;
08    //ponteiro para ponteiro de inteiro (2 níveis)
09    int **p2;
10    //ponteiro para ponteiro para ponteiro de inteiro(3 níveis)
11    int ***p3;
12    system("pause");
13    return 0;
14  }
```

Ponteiro para ponteiro

- Ex.:

```
char letra='a';
char *ptrChar;
char **ptrPtrChar;
char ***ptrPtr;
ptrChar = &letra;
ptrPtrChar = &ptrChar;
ptrPtr = &ptrPtrChar;
```

Memória		
#	var	conteúdo
119		
120	char ***ptrPtr	#122
121		
122	char **ptrPtrChar	#124
123		
124	char *ptrChar	#126
125		
126	char letra	‘a’
127		

Ponteiro para Funções

- O uso de ponteiros para funções em C serve principalmente para definir, em tempo de execução, qual função será executada, sem a necessidade de escrever o nome da função de forma explícita naquele ponto do código
- Em geral, a definição de qual função será executada é feita em um outro ponto do código, fazendo o ponteiro apontar para a função desejada.
- Inicialmente, se deve criar um tipo de dado que aponte para uma função, da seguinte forma:

```
typedef int TIPOFuncaoSemParametros();
typedef int TIPOFuncaoComParametros (int x, float y);
```

Exemplo básico:

- Vscode